```
//// FILE: backend/app/main.py  (310 lines)

"""GeM BidVerify AI - HTTP API.

Run from the backend folder:   uvicorn app.main:app --reload --port 8000
Interactive docs:              http://localhost:8000/docs
All endpoints live under /api and (except /api/auth/* and /api/health) need a Bearer token.
"""
from typing import List, Literal, Optional

from fastapi import Depends, FastAPI, File, Form, Header, Query, Request, UploadFile
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import FileResponse, JSONResponse
from pydantic import BaseModel

from . import db, services
from .services import ServiceError

app = FastAPI(title="GeM BidVerify AI", version="2.0.0",
              description="AI-assisted bid compliance verification for GeM procurement officers.")
# The API uses bearer tokens (no cookies), so a permissive CORS policy is safe for local use.
app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_methods=["*"], allow_headers=["*"])


@app.on_event("startup")
def _startup():
    db.init_db()


@app.exception_handler(ServiceError)
async def _service_error(_: Request, exc: ServiceError):
    return JSONResponse(status_code=exc.status, content={"detail": exc.message})


def current_user(authorization: Optional[str] = Header(None)) -> dict:
    if not authorization or not authorization.lower().startswith("bearer "):
        raise ServiceError(401, "Authentication required.")
    return services.user_from_token(authorization.split(" ", 1)[1].strip())


def require_officer(user: dict = Depends(current_user)) -> dict:
    role = (user.get("role") or "").strip()
    ALLOWED_OFFICER_ROLES = {"Procurement Officer", "Evaluation Cell Lead", "Vigilance Officer", "admin", "Super
     Admin"}
    if role not in ALLOWED_OFFICER_ROLES:
        raise ServiceError(403, "Access denied: This operation requires an authorized Procurement Officer role."
    )
    return user



# ---------------------------------------------------------------------------- request models
class LoginIn(BaseModel):
    username: str
    password: str


class OtpRequestIn(BaseModel):
    username: Optional[str] = None
    email: Optional[str] = None


class Verify2FAIn(BaseModel):
    username: Optional[str] = None
    email: Optional[str] = None
    otp: str


class ResetPasswordIn(BaseModel):
    username: Optional[str] = None
    email: Optional[str] = None
    otp: str
    new_password: str


class RegisterIn(BaseModel):
    username: str
    full_name: Optional[str] = None
    email: str
```

```
    password: str
    department: Optional[str] = None
    phone: Optional[str] = None


class RequirementIn(BaseModel):
    requirement_code: str
    requirement_name: str
    is_mandatory: bool = True
    verification_source: Optional[str] = None
    min_condition: Optional[str] = None
    weight: float = 10
    applicable_bidder_type: Optional[str] = "ALL"


class TenderIn(BaseModel):
    tender_id: Optional[str] = None
    title: str
    department: Optional[str] = ""
    tender_value: Optional[float] = None
    deadline: Optional[str] = None
    description: Optional[str] = ""
    requirements: List[RequirementIn] = []


class BidderIn(BaseModel):
    company_name: str
    cin: Optional[str] = None
    pan: Optional[str] = ""
    gstin: Optional[str] = None
    registered_address: Optional[str] = None
    contact_email: Optional[str] = None
    contact_phone: Optional[str] = None
    is_msme: bool = False
    msme_reg_no: Optional[str] = None
    is_startup: bool = False
    startup_dipp_no: Optional[str] = None


class BidIn(BaseModel):
    tender_id: int
    bidder_id: int


class ComplianceUpdateIn(BaseModel):
    points_awarded: float
    status: Literal["VERIFIED", "MISMATCH", "NON_COMPLIANT"]
    explanation: Optional[str] = None


class DecisionIn(BaseModel):
    decision: Literal["APPROVED", "REJECTED", "REQUIRES_CLARIFICATION"]
    comments: Optional[str] = ""
    clarification_details: Optional[str] = None


def _dump(model) -> dict:
    return model.model_dump() if hasattr(model, "model_dump") else model.dict()


# ---------------------------------------------------------------------------- health + auth
@app.get("/api/health")
def health():
    from . import extraction
    return {"status": "ok", "ocr_available": extraction.ocr_available()}


@app.post("/api/auth/login")
def login(body: LoginIn, request: Request):
    return services.login(body.username, body.password, request.client.host if request.client else "unknown")


@app.post("/api/auth/send-otp")
def send_otp(body: OtpRequestIn):
    return services.send_otp(body.username, body.email)


@app.post("/api/auth/verify-2fa")
def verify_2fa(body: Verify2FAIn):
```

```
    return services.verify_otp(body.username, body.email, body.otp)


@app.post("/api/auth/reset-password")
def reset_password(body: ResetPasswordIn):
    return services.reset_password(body.username, body.email, body.otp, body.new_password)


@app.post("/api/auth/register")
def register(body: RegisterIn, admin_user: dict = Depends(require_officer)):
    return services.register(_dump(body), authorized_by=admin_user)


@app.get("/api/auth/me")
def me(user: dict = Depends(current_user)):
    return services._public_user(user)


# ---------------------------------------------------------------------------- dashboard
@app.get("/api/dashboard")
def dashboard(user: dict = Depends(require_officer)):
    return services.dashboard()


# ---------------------------------------------------------------------------- tenders / bidders
@app.get("/api/tenders")
def list_tenders(user: dict = Depends(require_officer)):
    return services.list_tenders()


@app.get("/api/tenders/{tender_ref}")
def get_tender(tender_ref: str, user: dict = Depends(require_officer)):
    return services.get_tender(tender_ref)


@app.post("/api/tenders")
def create_tender(body: TenderIn, user: dict = Depends(require_officer)):
    return services.create_tender(_dump(body), user)


@app.post("/api/tenders/{tender_id}/requirements")
def add_requirement(tender_id: int, body: RequirementIn, user: dict = Depends(require_officer)):
    return services.add_requirement(tender_id, _dump(body), user)


@app.get("/api/bidders")
def list_bidders(user: dict = Depends(require_officer)):
    return services.list_bidders()


@app.post("/api/bidders")
def create_bidder(body: BidderIn, user: dict = Depends(require_officer)):
    return services.create_bidder(_dump(body), user)


# ---------------------------------------------------------------------------- bids
@app.get("/api/bids")
def list_bids(tender_id: Optional[int] = Query(None), user: dict = Depends(require_officer)):
    return services.list_bids(tender_id)


@app.get("/api/bids/{bid_ref}")
def get_bid(bid_ref: str, user: dict = Depends(require_officer)):
    return services.get_bid(bid_ref)


@app.post("/api/bids")
def create_bid(body: BidIn, user: dict = Depends(require_officer)):
    return services.create_bid(body.tender_id, body.bidder_id, user)


@app.post("/api/bids/{bid_ref}/documents")
def upload_document(bid_ref: str, file: UploadFile = File(...), classified_type_hint: Optional[str] = Form(None)
    ,
                    user: dict = Depends(require_officer)):
    return services.upload_document(bid_ref, file.filename, file.file.read(), classified_type_hint, user)


@app.delete("/api/bids/{bid_ref}/documents/{doc_id}")
```

```
def delete_document(bid_ref: str, doc_id: int, user: dict = Depends(require_officer)):
    return services.delete_document(bid_ref, doc_id, user)


@app.get("/api/documents/{doc_id}/file")
def download_document(doc_id: int, user: dict = Depends(require_officer)):
    path, name = services.document_file(doc_id)
    return FileResponse(path, filename=name)


@app.put("/api/bids/{bid_ref}/compliance/{result_id}")
def update_compliance(bid_ref: str, result_id: int, body: ComplianceUpdateIn, user: dict = Depends(require_offic
    er)):
    return services.update_compliance_result(bid_ref, result_id, _dump(body), user)


@app.post("/api/bids/{bid_ref}/verify")
def verify_bid(bid_ref: str, user: dict = Depends(require_officer)):
    return services.verify_bid(bid_ref, user)


@app.post("/api/bids/{bid_ref}/decision")
def decision(bid_ref: str, body: DecisionIn, user: dict = Depends(require_officer)):
    return services.submit_decision(bid_ref, _dump(body), user)


# ---------------------------------------------------------------------------- integrations (simulated)
@app.get("/api/integrations/status")
def integrations_status(user: dict = Depends(require_officer)):
    return services.integrations_status()


@app.post("/api/integrations/test-gst")
def test_gst(gstin: str, user: dict = Depends(require_officer)):
    from . import adapters
    return adapters.lookup_gst(gstin)


@app.post("/api/integrations/test-pan")
def test_pan(pan: str, user: dict = Depends(require_officer)):
    from . import adapters
    return adapters.lookup_pan(pan)


@app.post("/api/integrations/test-debarment")
def test_debarment(company_name: str, pan: str = "", user: dict = Depends(require_officer)):
    from . import adapters
    return adapters.check_debarment(company_name, pan)


# ---------------------------------------------------------------------------- audit / notifications / admin
@app.get("/api/audit")
def audit(user: dict = Depends(require_officer)):
    return services.audit_logs()


@app.get("/api/audit/verify")
def audit_verify(user: dict = Depends(require_officer)):
    return services.audit_verify()


@app.get("/api/notifications")
def notifications(user: dict = Depends(require_officer)):
    return services.notifications()


@app.post("/api/notifications/{notification_id}/read")
def read_notification(notification_id: int, user: dict = Depends(require_officer)):
    return services.mark_notification_read(notification_id)


@app.get("/api/admin/export")
def export_data(user: dict = Depends(require_officer)):
    return services.export_data()


@app.post("/api/admin/reset-demo")
def reset_demo(user: dict = Depends(require_officer)):
    return services.reset_demo(user)
```

```
//// FILE: backend/app/config.py  (60 lines)

"""Runtime configuration.

Everything can be overridden with environment variables; the defaults are safe for a
local demo (SQLite file next to the code, OTP shown in the console).
"""
import os
import secrets
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent
DATA_DIR = Path(os.getenv("GEM_DATA_DIR", BASE_DIR / "data"))
UPLOAD_DIR = DATA_DIR / "uploads"
DB_PATH = Path(os.getenv("GEM_DB_PATH", DATA_DIR / "gem_bidverify.db"))
for _p in (DATA_DIR, UPLOAD_DIR):
    _p.mkdir(parents=True, exist_ok=True)


def _load_secret() -> str:
    """JWT signing key: from GEM_SECRET_KEY, else generated once and kept in data/secret.key."""
    env = os.getenv("GEM_SECRET_KEY")
    if env:
        return env
    key_file = DATA_DIR / "secret.key"
    if not key_file.exists():
        key_file.write_text(secrets.token_hex(32))
    return key_file.read_text().strip()


SECRET_KEY = _load_secret()
TOKEN_TTL_HOURS = 8

OTP_TTL_SECONDS = 300
OTP_MAX_ATTEMPTS = 5
LOGIN_MAX_FAILURES = 5
LOGIN_LOCK_SECONDS = 300
MIN_PASSWORD_LENGTH = 6

MAX_UPLOAD_MB = 15
ALLOWED_EXTENSIONS = {".pdf", ".png", ".jpg", ".jpeg", ".tif", ".tiff", ".txt"}

# E-mail (optional). Without SMTP_HOST the OTP is printed in the server console.
SMTP_HOST = os.getenv("GEM_SMTP_HOST", "")
SMTP_PORT = int(os.getenv("GEM_SMTP_PORT", "587"))
SMTP_USER = os.getenv("GEM_SMTP_USER", "")
SMTP_PASSWORD = os.getenv("GEM_SMTP_PASSWORD", "")
SMTP_FROM = os.getenv("GEM_SMTP_FROM", SMTP_USER or "noreply@example.gov.in")
# For local demos the OTP is also returned to the browser when no mail server is set up.
SHOW_OTP_IN_RESPONSE = os.getenv("GEM_SHOW_OTP", "false" if SMTP_HOST else "true").lower() == "true"

DEMO_OFFICER_EMAIL = os.getenv("GEM_DEMO_OFFICER_EMAIL", "officer@example.gov.in")

# OCR
OCR_LANG = os.getenv("GEM_OCR_LANG", "eng")
TESSERACT_CMD = os.getenv("TESSERACT_CMD", "")
_WIN_TESSERACT = Path(r"C:\Program Files\Tesseract-OCR\tesseract.exe")
if not TESSERACT_CMD and _WIN_TESSERACT.exists():
    TESSERACT_CMD = str(_WIN_TESSERACT)

# If true a GSTIN whose check digit is wrong is treated as a failure (real GSTINs always pass).
STRICT_GSTIN_CHECKSUM = os.getenv("GEM_STRICT_GSTIN", "false").lower() == "true"


//// FILE: backend/app/db.py  (225 lines)

"""SQLite storage: schema, small helpers, hash-chained audit log, first-run demo data."""
import hashlib
import json
import sqlite3
from contextlib import contextmanager
from datetime import datetime, timezone
from pathlib import Path

from . import config, security

SCHEMA = """
```

```
CREATE TABLE IF NOT EXISTS users (
  id INTEGER PRIMARY KEY AUTOINCREMENT, username TEXT UNIQUE NOT NULL, full_name TEXT NOT NULL,
  role TEXT NOT NULL, department TEXT, email TEXT, phone TEXT, password_hash TEXT NOT NULL,
  is_active INTEGER NOT NULL DEFAULT 1);
CREATE TABLE IF NOT EXISTS otps (
  key TEXT PRIMARY KEY, otp_hash TEXT NOT NULL, email TEXT, expires_at REAL NOT NULL, attempts INTEGER NOT NULL
    DEFAULT 0);
CREATE TABLE IF NOT EXISTS tenders (
  id INTEGER PRIMARY KEY AUTOINCREMENT, tender_id TEXT UNIQUE NOT NULL, title TEXT NOT NULL, department TEXT,
  tender_value REAL, deadline TEXT, status TEXT NOT NULL DEFAULT 'ACTIVE', description TEXT, created_at TEXT NOT
     NULL);
CREATE TABLE IF NOT EXISTS requirements (
  id INTEGER PRIMARY KEY AUTOINCREMENT, tender_pk INTEGER NOT NULL REFERENCES tenders(id) ON DELETE CASCADE,
  requirement_code TEXT NOT NULL, requirement_name TEXT NOT NULL, is_mandatory INTEGER NOT NULL DEFAULT 1,
  verification_source TEXT, min_condition TEXT, weight REAL NOT NULL DEFAULT 10, applicable_bidder_type TEXT DEF
    AULT 'ALL');
CREATE TABLE IF NOT EXISTS bidders (
  id INTEGER PRIMARY KEY AUTOINCREMENT, company_name TEXT NOT NULL, cin TEXT, pan TEXT, gstin TEXT,
  registered_address TEXT, contact_email TEXT, contact_phone TEXT, is_msme INTEGER DEFAULT 0, msme_reg_no TEXT,
  is_startup INTEGER DEFAULT 0, startup_dipp_no TEXT);
CREATE TABLE IF NOT EXISTS bids (
  id INTEGER PRIMARY KEY AUTOINCREMENT, bid_id TEXT UNIQUE NOT NULL,
  tender_pk INTEGER NOT NULL REFERENCES tenders(id), bidder_pk INTEGER NOT NULL REFERENCES bidders(id),
  submission_date TEXT NOT NULL, status TEXT NOT NULL, compliance_score INTEGER DEFAULT 0, risk_level TEXT DEFAU
    LT 'LOW',
  ai_recommendation TEXT, ai_explanation TEXT, risk_score INTEGER DEFAULT 0, risk_factors TEXT DEFAULT '[]',
  recommended_action TEXT, tender_id TEXT, vendor_name TEXT, gstin TEXT, file_paths TEXT);
CREATE TABLE IF NOT EXISTS vendors (
  id INTEGER PRIMARY KEY AUTOINCREMENT, company_name TEXT NOT NULL, gstin TEXT UNIQUE NOT NULL,
  email TEXT UNIQUE NOT NULL, password_hash TEXT NOT NULL, pan TEXT, contact_phone TEXT,
  business_type TEXT DEFAULT 'REGULAR', address TEXT, created_at TEXT NOT NULL);

CREATE TABLE IF NOT EXISTS documents (
  id INTEGER PRIMARY KEY AUTOINCREMENT, bid_pk INTEGER NOT NULL REFERENCES bids(id) ON DELETE CASCADE,
  file_name TEXT NOT NULL, file_type TEXT, file_size_kb INTEGER, stored_path TEXT, classified_type TEXT,
  classification_confidence REAL, ocr_status TEXT, extracted_fields TEXT DEFAULT '[]', created_at TEXT NOT NULL)
    ;
CREATE TABLE IF NOT EXISTS compliance_results (
  id INTEGER PRIMARY KEY AUTOINCREMENT, bid_pk INTEGER NOT NULL REFERENCES bids(id) ON DELETE CASCADE,
  requirement_code TEXT NOT NULL, requirement_name TEXT, status TEXT NOT NULL, points_awarded REAL, max_points R
    EAL,
  evidence_document TEXT, confidence_score REAL, rule_applied TEXT, explanation TEXT, flags TEXT DEFAULT '[]',
  is_mandatory INTEGER DEFAULT 1, officer_override INTEGER DEFAULT 0);
CREATE TABLE IF NOT EXISTS decisions (
  id INTEGER PRIMARY KEY AUTOINCREMENT, bid_pk INTEGER UNIQUE NOT NULL REFERENCES bids(id) ON DELETE CASCADE,
  officer_name TEXT, decision TEXT NOT NULL, comments TEXT, clarification_details TEXT, decided_at TEXT NOT NULL
    );
CREATE TABLE IF NOT EXISTS audit_logs (
  id INTEGER PRIMARY KEY AUTOINCREMENT, bid_id INTEGER, user_name TEXT, action TEXT NOT NULL, entity_type TEXT,
  details TEXT, rule_applied TEXT, ip_address TEXT, timestamp TEXT NOT NULL, prev_hash TEXT, entry_hash TEXT);
CREATE TABLE IF NOT EXISTS notifications (
  id INTEGER PRIMARY KEY AUTOINCREMENT, title TEXT, message TEXT, category TEXT, is_read INTEGER DEFAULT 0,
  link TEXT, created_at TEXT NOT NULL);
CREATE TABLE IF NOT EXISTS connectors (
  id TEXT PRIMARY KEY, name TEXT, type TEXT, mode TEXT, status TEXT, latency_ms INTEGER, badge TEXT);
CREATE TABLE IF NOT EXISTS evaluations (
  id INTEGER PRIMARY KEY AUTOINCREMENT, bid_pk INTEGER NOT NULL, evaluated_at TEXT NOT NULL, score INTEGER,
  risk_level TEXT, recommendation TEXT);
"""


def now_iso() -> str:
    return datetime.now(timezone.utc).isoformat(timespec="milliseconds").replace("+00:00", "Z")


@contextmanager
def connect():
    conn = sqlite3.connect(config.DB_PATH, timeout=30)
    conn.row_factory = sqlite3.Row
    conn.execute("PRAGMA foreign_keys=ON")
    try:
        yield conn
        conn.commit()
    except Exception:
        conn.rollback()
        raise
    finally:
        conn.close()
```

```
def rows(conn, sql, params=()):
    return [dict(r) for r in conn.execute(sql, params).fetchall()]


def row(conn, sql, params=()):
    r = conn.execute(sql, params).fetchone()
    return dict(r) if r else None


def loads(value, default):
    try:
        return json.loads(value) if value else default
    except (TypeError, ValueError):
        return default


# ---------------------------------------------------------------------------------------
# Tamper-evident audit log: every entry stores SHA-256(previous hash + entry content).
# Changing or deleting an old row breaks the chain, which verify_audit_chain() detects.
# ---------------------------------------------------------------------------------------
def _entry_hash(prev_hash: str, entry: dict) -> str:
    payload = json.dumps(entry, sort_keys=True, ensure_ascii=False)
    return hashlib.sha256((prev_hash + payload).encode()).hexdigest()


def add_audit(conn, action, details, entity_type="BID", bid_id=None, user_name="System",
              rule_applied=None, ip_address="127.0.0.1", timestamp=None):
    last = row(conn, "SELECT entry_hash FROM audit_logs ORDER BY id DESC LIMIT 1")
    prev = last["entry_hash"] if last and last["entry_hash"] else "GENESIS"
    entry = {"bid_id": bid_id, "user_name": user_name, "action": action, "entity_type": entity_type,
             "details": details, "rule_applied": rule_applied, "ip_address": ip_address,
             "timestamp": timestamp or now_iso()}
    conn.execute(
        "INSERT INTO audit_logs (bid_id,user_name,action,entity_type,details,rule_applied,ip_address,timestamp,p
    rev_hash,entry_hash)"
        " VALUES (:bid_id,:user_name,:action,:entity_type,:details,:rule_applied,:ip_address,:timestamp,:prev,:h
    )",
        {**entry, "prev": prev, "h": _entry_hash(prev, entry)})


def verify_audit_chain(conn) -> dict:
    prev, count = "GENESIS", 0
    for r in conn.execute("SELECT * FROM audit_logs ORDER BY id"):
        r = dict(r)
        entry = {k: r[k] for k in ("bid_id", "user_name", "action", "entity_type", "details",
                                   "rule_applied", "ip_address", "timestamp")}
        if r["prev_hash"] != prev or r["entry_hash"] != _entry_hash(prev, entry):
            return {"valid": False, "entries_checked": count, "first_broken_id": r["id"]}
        prev, count = r["entry_hash"], count + 1
    return {"valid": True, "entries_checked": count, "first_broken_id": None}


# ---------------------------------------------------------------------------------------
# Schema + demo data
# ---------------------------------------------------------------------------------------
def init_db():
    with connect() as conn:
        conn.executescript(SCHEMA)
        for col in ("tender_id", "vendor_name", "gstin", "file_paths"):
            try:
                conn.execute(f"ALTER TABLE bids ADD COLUMN {col} TEXT")
            except sqlite3.OperationalError:
                pass
        if not row(conn, "SELECT id FROM users LIMIT 1"):
            seed_demo_data(conn)



def _b(v):
    return 1 if v else 0


def seed_demo_data(conn):
    """Load the demo tenders, bidders, bids and users that ship with the project."""
    seeds = json.loads((Path(__file__).parent / "seed_data" / "seeds.json").read_text(encoding="utf-8"))

    for u in seeds["seedUsers"]:
        conn.execute(
```

```
            "INSERT INTO users (id,username,full_name,role,department,email,phone,password_hash,is_active) VALUE
    S (?,?,?,?,?,?,?,?,1)",
            (u["id"], u["username"], u["full_name"], u["role"], u["department"], config.DEMO_OFFICER_EMAIL,
             u.get("phone"), security.hash_password(u["password"])))

    for t in seeds["seedTenders"]:
        conn.execute(
            "INSERT INTO tenders (id,tender_id,title,department,tender_value,deadline,status,description,created
    _at) VALUES (?,?,?,?,?,?,?,?,?)",
            (t["id"], t["tender_id"], t["title"], t["department"], t["tender_value"], t["deadline"], t["status"]
    ,
             t.get("description"), t["created_at"]))
        for r in t["requirements"]:
            conn.execute(
                "INSERT INTO requirements (tender_pk,requirement_code,requirement_name,is_mandatory,verification
    _source,min_condition,weight,applicable_bidder_type) VALUES (?,?,?,?,?,?,?,?)",
                (t["id"], r["requirement_code"], r["requirement_name"], _b(r["is_mandatory"]), r["verification_s
    ource"],
                 r["min_condition"], r["weight"], r.get("applicable_bidder_type", "ALL")))

    for b in seeds["seedBidders"]:
        conn.execute(
            "INSERT INTO bidders (id,company_name,cin,pan,gstin,registered_address,contact_email,contact_phone,i
    s_msme,msme_reg_no,is_startup,startup_dipp_no) VALUES (?,?,?,?,?,?,?,?,?,?,?,?)",
            (b["id"], b["company_name"], b.get("cin"), b.get("pan"), b.get("gstin"), b.get("registered_address")
    ,
             b.get("contact_email"), b.get("contact_phone"), _b(b.get("is_msme")), b.get("msme_reg_no"),
             _b(b.get("is_startup")), b.get("startup_dipp_no")))

    for bid in seeds["seedBids"]:
        ra = bid["risk_assessment"]
        conn.execute(
            "INSERT INTO bids (id,bid_id,tender_pk,bidder_pk,submission_date,status,compliance_score,risk_level,
    ai_recommendation,ai_explanation,risk_score,risk_factors,recommended_action) VALUES (?,?,?,?,?,?,?,?,?,?,?
    ,?)",
            (bid["id"], bid["bid_id"], bid["tender_id"], bid["bidder_id"], bid["submission_date"], bid["status"]
    ,
             bid["compliance_score"], bid["risk_level"], bid["ai_recommendation"], bid["ai_explanation"],
             ra["risk_score"], json.dumps(ra["risk_factors"]), ra["recommended_action"]))
        for d in bid["documents"]:
            conn.execute(
                "INSERT INTO documents (id,bid_pk,file_name,file_type,file_size_kb,stored_path,classified_type,c
    lassification_confidence,ocr_status,extracted_fields,created_at) VALUES (?,?,?,?,?,?,?,?,?,?,?)",
                (d["id"], bid["id"], d["file_name"], d["file_type"], d["file_size_kb"], None, d["classified_type
    "],
                 d["classification_confidence"], d["ocr_status"], json.dumps(d["extracted_fields"]), d["created_
    at"]))
        for c in bid["compliance_results"]:
            conn.execute(
                "INSERT INTO compliance_results (id,bid_pk,requirement_code,requirement_name,status,points_award
    ed,max_points,evidence_document,confidence_score,rule_applied,explanation,flags) VALUES (?,?,?,?,?,?,?,?,?,?
    ,?,'[]')",
                (c["id"], bid["id"], c["requirement_code"], c["requirement_name"], c["status"], c["points_awarde
    d"],
                 c["max_points"], c["evidence_document"], c["confidence_score"], c["rule_applied"], c["explanati
    on"]))
        conn.execute("INSERT INTO evaluations (bid_pk,evaluated_at,score,risk_level,recommendation) VALUES (?,?,
    ?,?,?)",
                     (bid["id"], bid["submission_date"], bid["compliance_score"], bid["risk_level"], bid["ai_rec
    ommendation"]))

    for a in seeds["seedAuditLogs"]:
        add_audit(conn, a["action"], a["details"], a.get("entity_type", "BID"), None, a["user_name"],
                  a.get("rule_applied"), a.get("ip_address", "127.0.0.1"), a["timestamp"])
    for n in seeds["seedNotifications"]:
        conn.execute("INSERT INTO notifications (id,title,message,category,is_read,link,created_at) VALUES (?,?,
    ?,?,?,?,?)",
                     (n["id"], n["title"], n["message"], n["category"], _b(n["is_read"]), n.get("link"), n["crea
    ted_at"]))
    for c in seeds["seedConnectors"]:
        conn.execute("INSERT INTO connectors (id,name,type,mode,status,latency_ms,badge) VALUES (?,?,?,?,?,?,?)"
    ,
                     (c["id"], c["name"], c["type"], c["mode"], c["status"], c["latency_ms"], "Simulated"))


def reset_demo_data():
    """Drop everything and reload the demo data (uploaded files are removed as well)."""
    with connect() as conn:
        for table in ("evaluations", "decisions", "compliance_results", "documents", "bids", "requirements",
```

```
                    "tenders", "bidders", "audit_logs", "notifications", "connectors", "otps", "users"):
            conn.execute(f"DELETE FROM {table}")
        conn.execute("DELETE FROM sqlite_sequence")
        seed_demo_data(conn)
        add_audit(conn, "DATA_RESET_DEFAULT", "Demo data restored to the shipped baseline.", "SYSTEM", None, "Sy
    stem")
    for f in config.UPLOAD_DIR.glob("**/*"):
        if f.is_file():
            f.unlink()


//// FILE: backend/app/security.py  (66 lines)

"""Password hashing, JWT tokens and one-time passwords (standard library + PyJWT)."""
import base64
import hashlib
import hmac
import secrets
import time

import jwt

from . import config

_ITERATIONS = 200_000


def hash_password(password: str) -> str:
    salt = secrets.token_bytes(16)
    digest = hashlib.pbkdf2_hmac("sha256", password.encode(), salt, _ITERATIONS)
    return "pbkdf2_sha256${}${}${}".format(
        _ITERATIONS, base64.b64encode(salt).decode(), base64.b64encode(digest).decode()
    )


def verify_password(password: str, stored: str) -> bool:
    try:
        _, iterations, salt_b64, digest_b64 = stored.split("$")
        salt = base64.b64decode(salt_b64)
        expected = base64.b64decode(digest_b64)
        actual = hashlib.pbkdf2_hmac("sha256", password.encode(), salt, int(iterations))
        return hmac.compare_digest(actual, expected)
    except (ValueError, TypeError):
        return False


def create_token(user: dict) -> str:
    now = int(time.time())
    payload = {
        "sub": str(user["id"]),
        "username": user["username"],
        "role": user.get("role", "Procurement Officer"),
        "iat": now,
        "exp": now + config.TOKEN_TTL_HOURS * 3600,
    }
    return jwt.encode(payload, config.SECRET_KEY, algorithm="HS256")



def decode_token(token: str) -> dict:
    """Return the token payload or raise jwt.InvalidTokenError."""
    return jwt.decode(token, config.SECRET_KEY, algorithms=["HS256"])


def generate_otp() -> str:
    return str(100000 + secrets.randbelow(900000))


def hash_otp(otp: str) -> str:
    return hmac.new(config.SECRET_KEY.encode(), otp.encode(), hashlib.sha256).hexdigest()


def mask_email(email: str) -> str:
    if "@" not in email:
        return email
    local, domain = email.split("@", 1)
    if len(local) <= 2:
        return f"{local}*@{domain}"
    return local[:2] + "*" * max(2, len(local) - 3) + local[-1] + "@" + domain
```

```
//// FILE: backend/app/services.py  (691 lines)

"""Business logic. Every function returns plain dictionaries shaped exactly like the frontend expects."""
import json
import random
import re
import smtplib
import time
import uuid
from datetime import datetime, timedelta, timezone
from email.message import EmailMessage
from pathlib import Path

from . import adapters, config, db, extraction, rules, security
from .db import connect, loads, now_iso, row, rows
from .extraction import PAN_RE, GSTIN_RE


class ServiceError(Exception):
    def __init__(self, status: int, message: str):
        super().__init__(message)
        self.status, self.message = status, message


def _num(x):
    """JSON-friendly number: 10.0 -> 10, 7.5 -> 7.5"""
    if x is None:
        return None
    return int(x) if float(x).is_integer() else round(float(x), 2)


# ============================================================================================
# Authentication
# ============================================================================================
_login_failures = {}  # username -> (count, locked_until)


def _public_user(u: dict) -> dict:
    return {"id": u["id"], "username": u["username"], "full_name": u["full_name"], "role": u["role"],
            "department": u["department"], "email": u["email"], "phone": u["phone"], "is_active": bool(u["is_act
    ive"])}


def _session(u: dict) -> dict:
    return {"access_token": security.create_token(u), "token_type": "bearer", "user": _public_user(u)}


def _find_user(conn, username=None, email=None):
    if username:
        u = row(conn, "SELECT * FROM users WHERE username=? OR lower(email)=lower(?)", (username, username))
        if u:
            return u
    if email:
        return row(conn, "SELECT * FROM users WHERE lower(email)=lower(?)", (email,))
    return None


def user_from_token(token: str) -> dict:
    import jwt
    try:
        payload = security.decode_token(token)
    except jwt.InvalidTokenError:
        raise ServiceError(401, "Session expired or invalid. Please sign in again.")
    with connect() as conn:
        u = row(conn, "SELECT * FROM users WHERE id=? AND is_active=1", (int(payload["sub"]),))
    if not u:
        raise ServiceError(401, "Account not found or disabled.")
    # Verify token role matches database user role
    token_role = payload.get("role")
    if token_role and token_role != u["role"]:
        raise ServiceError(403, "Access denied: Token role does not match user account.")
    return u


def login(username: str, password: str, ip: str = "unknown") -> dict:
    count, locked_until = _login_failures.get(username, (0, 0))
    if locked_until > time.time():
```

```
                <p>Digital Signature Certificate (DSC) / SHA-256 Audit Seal embedded.</p>
            </div>
            <div class="text-center font-mono">
                <div class="w-24 h-24 border-2 border-dashed border-slate-300 rounded-xl flex items-center justi
    fy-center text-[10px] text-slate-400 mx-auto mb-1">
                    [ GeM e-Seal ]
                </div>
                <span class="text-[10px] font-bold text-slate-700">SPV Verified</span>
            </div>
        </div>

    </div>

</body>
</html>


//// FILE: vendor_portal/templates/register.html  (201 lines)

{% extends "base.html" %}

{% block title %}Vendor Registration - GeM e-Procurement Portal{% endblock %}

{% block content %}
<div class="max-w-2xl mx-auto">

    <!-- Header Breadcrumb & Intro -->
    <div class="mb-6 text-center sm:text-left">
        <div class="flex items-center justify-center sm:justify-start gap-2 text-xs text-slate-500 mb-2">
            <a href="/tenders" class="hover:text-gem-700">GeM Portal</a>
            <span>/</span>
            <span class="text-slate-800 font-semibold">Vendor Onboarding</span>
        </div>
        <h1 class="text-2xl sm:text-3xl font-extrabold text-slate-900 tracking-tight">
            Register as an Approved Vendor
        </h1>
        <p class="text-xs sm:text-sm text-slate-500 mt-1">
            Create your statutory corporate profile to participate in Central and State Government e-Tenders on
    GeM.
        </p>
    </div>

    <!-- Quick Demo Fill Banner -->
    <div class="bg-gradient-to-r from-amber-50 to-orange-50 border border-amber-200/90 rounded-2xl p-4 mb-6 shad
    ow-sm flex flex-col sm:flex-row items-center justify-between gap-3">
        <div class="flex items-center gap-3">
            <div class="w-8 h-8 rounded-lg bg-amber-500 text-white flex items-center justify-center font-bold te
    xt-sm shadow-sm flex-shrink-0">

            </div>
            <div>
                <div class="text-xs font-bold text-amber-950">Project Review / Professor Demo Shortcut</div>
                <div class="text-[11px] text-amber-800">Fill authentic sample enterprise data with a single clic
    k.</div>
            </div>
        </div>
        <button type="button" onclick="fillDemoVendor()" class="px-3.5 py-1.5 bg-amber-600 hover:bg-amber-700 te
    xt-white text-xs font-bold rounded-xl shadow-sm transition-all flex items-center gap-1.5 flex-shrink-0">
            <span>  Auto-Fill Demo Enterprise</span>
        </button>
    </div>

    <!-- Main Registration Form Card -->
    <div class="bg-white rounded-2xl border border-slate-200/90 shadow-card p-6 sm:p-8">
        <form action="/register" method="POST" class="space-y-5" id="registerForm">

            <!-- Company Legal Name -->
            <div>
                <label for="company_name" class="block text-xs font-bold text-slate-700 mb-1.5">
                    Company / Enterprise Legal Name <span class="text-rose-500">*</span>
                </label>
                <div class="relative">
                    <input type="text" id="company_name" name="company_name" required
                           placeholder="e.g., Bharat Cybernetics & Cloud Solutions Pvt Ltd"
                           class="w-full text-xs sm:text-sm border border-slate-300 rounded-xl px-4 py-3 focus:r
    ing-2 focus:ring-gem-600 focus:border-gem-600 focus:outline-none transition-all placeholder:text-slate-400">
                </div>
                <p class="text-[11px] text-slate-500 mt-1">As registered with Ministry of Corporate Affairs (MCA
    ) / GSTN.</p>
```

```
        </div>

        <!-- GSTIN & Auto-Derived PAN Grid -->
        <div class="grid grid-cols-1 sm:grid-cols-2 gap-4">
            <div>
                <label for="gstin" class="block text-xs font-bold text-slate-700 mb-1.5">
                    GSTIN (15 Alphanumeric) <span class="text-rose-500">*</span>
                </label>
                <input type="text" id="gstin" name="gstin" required maxlength="15"
                       oninput="handleGstinInput(this)"
                       placeholder="e.g., 27AABCT3421K1Z5"
                       class="w-full text-xs sm:text-sm font-mono uppercase tracking-wider border border-sla
te-300 rounded-xl px-4 py-3 focus:ring-2 focus:ring-gem-600 focus:border-gem-600 focus:outline-none transiti
on-all placeholder:text-slate-400">
                <div id="gstinFeedback" class="text-[10px] text-slate-500 mt-1 font-mono">Format: 2 digits +
 10-char PAN + 1 + Z + Check Digit</div>
            </div>

            <div>
                <label for="pan" class="block text-xs font-bold text-slate-700 mb-1.5">
                    Permanent Account Number (PAN)
                </label>
                <input type="text" id="pan" name="pan" readonly
                       placeholder="Auto-extracted from GSTIN"
                       class="w-full text-xs sm:text-sm font-mono uppercase tracking-wider bg-slate-50 borde
r border-slate-200 text-slate-600 rounded-xl px-4 py-3 cursor-not-allowed">
                <div class="text-[10px] text-slate-500 mt-1 font-mono">Derived from GSTIN characters 3 to 12
</div>
            </div>
        </div>

        <!-- Official Email & Mobile -->
        <div class="grid grid-cols-1 sm:grid-cols-2 gap-4">
            <div>
                <label for="email" class="block text-xs font-bold text-slate-700 mb-1.5">
                    Official Correspondence Email <span class="text-rose-500">*</span>
                </label>
                <input type="email" id="email" name="email" required
                       placeholder="tender-desk@enterprise.co.in"
                       class="w-full text-xs sm:text-sm border border-slate-300 rounded-xl px-4 py-3 focus:r
ing-2 focus:ring-gem-600 focus:border-gem-600 focus:outline-none transition-all placeholder:text-slate-400">
            </div>

            <div>
                <label for="contact_phone" class="block text-xs font-bold text-slate-700 mb-1.5">
                    Primary Contact Number
                </label>
                <input type="tel" id="contact_phone" name="contact_phone"
                       placeholder="+91 98765 43210"
                       class="w-full text-xs sm:text-sm font-mono border border-slate-300 rounded-xl px-4 py
-3 focus:ring-2 focus:ring-gem-600 focus:border-gem-600 focus:outline-none transition-all placeholder:text-s
late-400">
            </div>
        </div>

        <!-- Password & Business Type -->
        <div class="grid grid-cols-1 sm:grid-cols-2 gap-4">
            <div>
                <label for="password" class="block text-xs font-bold text-slate-700 mb-1.5">
                    Portal Security Password <span class="text-rose-500">*</span>
                </label>
                <input type="password" id="password" name="password" required minlength="6"
                       placeholder="Minimum 6 characters"
                       class="w-full text-xs sm:text-sm border border-slate-300 rounded-xl px-4 py-3 focus:r
ing-2 focus:ring-gem-600 focus:border-gem-600 focus:outline-none transition-all">
            </div>

            <div>
                <label for="business_type" class="block text-xs font-bold text-slate-700 mb-1.5">
                    Enterprise Category (GFR Benefits)
                </label>
                <select id="business_type" name="business_type"
                        class="w-full text-xs sm:text-sm border border-slate-300 rounded-xl px-4 py-3 bg-whi
te focus:ring-2 focus:ring-gem-600 focus:border-gem-600 focus:outline-none transition-all font-medium">
                    <option value="REGULAR">Regular Enterprise</option>
                    <option value="MSME">Micro & Small Enterprise (MSME - Udyam)</option>
                    <option value="STARTUP">DIPP / DPIIT Recognized Startup</option>
                </select>
            </div>
```

```
            </div>

            <!-- Registered Address -->
            <div>
                <label for="address" class="block text-xs font-bold text-slate-700 mb-1.5">
                    Registered Corporate Address
                </label>
                <textarea id="address" name="address" rows="2"
                          placeholder="Plot 42, Electronics City Phase 1, Bengaluru, Karnataka - 560100"
                          class="w-full text-xs sm:text-sm border border-slate-300 rounded-xl px-4 py-2.5 focus:
    ring-2 focus:ring-gem-600 focus:border-gem-600 focus:outline-none transition-all placeholder:text-slate-400"
    ></textarea>
            </div>

            <!-- Terms & Statutory Confirmation -->
            <div class="pt-2 border-t border-slate-100">
                <label class="flex items-start gap-2.5 cursor-pointer">
                    <input type="checkbox" required checked class="mt-0.5 rounded text-gem-700 focus:ring-gem-60
    0 border-slate-300">
                    <span class="text-[11px] text-slate-600 leading-normal">
                        I hereby declare that the corporate details and statutory GSTIN provided are true and ac
    curate under the General Financial Rules (GFR 2017) Integrity Pact.
                    </span>
                </label>
            </div>

            <!-- Submit Button -->
            <button type="submit" class="w-full py-3.5 bg-gradient-to-r from-gem-700 to-gem-900 hover:from-gem-8
    00 hover:to-gem-950 text-white text-xs sm:text-sm font-bold rounded-xl shadow-md shadow-gem-900/20 hover:sha
    dow-lg transition-all flex items-center justify-center gap-2">
                <span>Complete Vendor Registration & Continue</span>
                <svg class="w-4 h-4" fill="none" stroke="currentColor" viewBox="0 0 24 24"><path stroke-linecap=
    "round" stroke-linejoin="round" stroke-width="2" d="M14 5l7 7m0 0l-7 7m7-7H3"></path></svg>
            </button>

            <!-- Already Registered Link -->
            <div class="text-center text-xs text-slate-500 pt-2">
                Already registered your enterprise?
                <a href="/login" class="text-gem-700 hover:text-gem-900 font-bold hover:underline">
                    Sign in to Vendor Portal &rarr;
                </a>
            </div>

        </form>
    </div>

</div>

<script>
function handleGstinInput(input) {
    const val = input.value.toUpperCase();
    input.value = val;
    const panInput = document.getElementById('pan');
    const feedback = document.getElementById('gstinFeedback');

    if (val.length >= 12) {
        panInput.value = val.substring(2, 12);
    } else {
        panInput.value = '';
    }

    const regex = /^[0-9]{2}[A-Z]{5}[0-9]{4}[A-Z]{1}[1-9A-Z]{1}Z[0-9A-Z]{1}$/;
    if (val.length === 15) {
        if (regex.test(val)) {
            feedback.innerHTML = '<span class="text-emerald-600 font-bold">  Valid GSTIN Structure Verified</spa
    n>';
        } else {
            feedback.innerHTML = '<span class="text-rose-600 font-bold">  Invalid GSTIN Pattern</span>';
        }
    } else {
        feedback.innerText = `${val.length}/15 characters entered`;
    }
}

function fillDemoVendor() {
    document.getElementById('company_name').value = "NexGen Compute Technologies Pvt Ltd";
    document.getElementById('gstin').value = "27AABCT3421K1Z5";
    document.getElementById('pan').value = "AABCT3421K";
    document.getElementById('email').value = "bids@nexgencompute.in";
```

```
    document.getElementById('contact_phone').value = "+91 98201 54321";
    document.getElementById('password').value = "Vendor@2026";
    document.getElementById('business_type').value = "MSME";
    document.getElementById('address').value = "Tower B, Cyber City, Phase II, Mumbai, Maharashtra - 400051";
    document.getElementById('gstinFeedback').innerHTML = '<span class="text-emerald-600 font-bold">  Valid Demo
    GSTIN Structure</span>';
}
</script>
{% endblock %}


//// FILE: vendor_portal/templates/success.html  (90 lines)

{% extends "base.html" %}

{% block title %}Bid Submitted Successfully - GeM Vendor Portal{% endblock %}

{% block content %}
<div class="max-w-2xl mx-auto py-4">

    <!-- Success Card -->
    <div class="bg-white rounded-3xl border border-slate-200/90 shadow-xl overflow-hidden text-center p-8 sm:p-1
    0 relative">
        <div class="w-16 h-16 rounded-2xl bg-emerald-100 text-emerald-600 flex items-center justify-center mx-au
    to mb-5 shadow-sm border border-emerald-200">
            <svg class="w-9 h-9" fill="none" stroke="currentColor" viewBox="0 0 24 24"><path stroke-linecap="rou
    nd" stroke-linejoin="round" stroke-width="2.5" d="M5 13l4 4L19 7"></path></svg>
        </div>

        <div class="inline-flex items-center gap-1.5 px-3 py-1 rounded-full bg-emerald-50 text-emerald-800 text-
    xs font-bold mb-3 border border-emerald-200">
            <span class="w-2 h-2 rounded-full bg-emerald-500"></span>
            SUBMISSION ACKNOWLEDGED & ENCRYPTED
        </div>

        <h1 class="text-2xl sm:text-3xl font-extrabold text-slate-900 tracking-tight">
            Tender Bid Submitted Successfully!
        </h1>
        <p class="text-xs sm:text-sm text-slate-500 mt-2 max-w-md mx-auto leading-relaxed">
            Your technical proposals and statutory compliance documents have been securely stored in the shared
    GeM database and queued for AI verification.
        </p>

        <!-- Acknowledgment Details Box -->
        <div class="mt-8 p-6 rounded-2xl bg-slate-50 border border-slate-200/90 text-left space-y-3 font-sans">
            <div class="flex items-center justify-between pb-3 border-b border-slate-200">
                <span class="text-xs font-bold text-slate-500 uppercase tracking-wider">Bid Reference Number</sp
    an>
                <span class="font-mono text-sm sm:text-base font-black text-gem-900 px-2 py-0.5 rounded bg-gem-1
    00 border border-gem-200">
                    {{ bid.bid_id }}
                </span>
            </div>

            <div class="flex items-center justify-between text-xs">
                <span class="text-slate-500 font-medium">Target Tender:</span>
                <span class="font-mono font-bold text-slate-800">{{ bid.tender_id }}</span>
            </div>

            <div class="flex items-center justify-between text-xs">
                <span class="text-slate-500 font-medium">Submitting Entity:</span>
                <span class="font-bold text-slate-800">{{ bid.vendor_name }}</span>
            </div>

            <div class="flex items-center justify-between text-xs">
                <span class="text-slate-500 font-medium">Entity GSTIN:</span>
                <span class="font-mono font-bold text-gem-800">{{ bid.gstin }}</span>
            </div>

            <div class="flex items-center justify-between text-xs">
                <span class="text-slate-500 font-medium">Documents Uploaded:</span>
                <span class="font-bold text-slate-800">{{ bid.documents_count }} Verified PDF(s)</span>
            </div>

            <div class="flex items-center justify-between text-xs pt-2 border-t border-slate-200">
                <span class="text-slate-500 font-medium">Verification Status:</span>
                <span class="inline-flex items-center gap-1.5 font-bold text-amber-700 bg-amber-50 px-2.5 py-0.5
     rounded-full border border-amber-200 text-[11px]">
                    <span class="w-1.5 h-1.5 rounded-full bg-amber-500 animate-pulse"></span>
```

```
                {{ bid.status }}
            </span>
        </div>
    </div>

    <!-- Submission Status Notice -->
    <div class="mt-6 p-4 rounded-xl bg-emerald-50/80 border border-emerald-200 text-xs text-emerald-950 text
-left flex items-start gap-3">
        <div class="text-base mt-0.5"> </div>
        <div class="leading-relaxed">
            <strong class="font-bold">Bid Submission Queued:</strong> Your bid (<code class="font-mono font-
bold">{{ bid.bid_id }}</code>) has been securely registered in the GeM procurement database. Attached techni
cal proposals, price schedules, and statutory certificates are queued for automated AI verification and comp
liance scoring.
        </div>
    </div>

    <!-- Action CTAs -->
    <div class="mt-8 flex flex-col sm:flex-row items-center justify-center gap-3">
        <a href="/my-bids" class="w-full sm:w-auto px-6 py-3 bg-gem-700 hover:bg-gem-800 text-white font-bol
d text-xs rounded-xl shadow-md transition-all flex items-center justify-center gap-2">
            <span>Track in My Submissions</span>
            <svg class="w-4 h-4" fill="none" stroke="currentColor" viewBox="0 0 24 24"><path stroke-linecap=
"round" stroke-linejoin="round" stroke-width="2" d="M14 5l7 7m0 0l-7 7m7-7H3"></path></svg>
        </a>
        <a href="/bids/{{ bid.id }}/receipt" class="w-full sm:w-auto px-5 py-3 bg-white hover:bg-slate-50 te
xt-gem-800 border border-gem-300 font-bold text-xs rounded-xl shadow-xs transition-colors flex items-center
justify-center gap-2">
            <svg class="w-4 h-4 text-gem-700" fill="none" stroke="currentColor" viewBox="0 0 24 24"><path st
roke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M17 17h2a2 2 0 002-2v-4a2 2 0 00-2-2H5a2 2
0 00-2 2v4a2 2 0 002 2h2m2 4h6a2 2 0 002-2v-4a2 2 0 00-2-2H9a2 2 0 00-2 2v4a2 2 0 002 2zm8-12V5a2 2 0 00-2-2
H9a2 2 0 00-2 2v4h10z"></path></svg>
            <span>View Official Receipt</span>
        </a>
        <a href="/tenders" class="w-full sm:w-auto px-4 py-3 text-slate-600 hover:text-slate-900 font-bold t
ext-xs rounded-xl transition-colors">
            Browse More Tenders
        </a>
    </div>

</div>
{% endblock %}


//// FILE: vendor_portal/templates/tender_detail.html  (124 lines)

{% extends "base.html" %}

{% block title %}{{ tender.tender_id }} - Tender Details - GeM Vendor Portal{% endblock %}

{% block content %}
<div class="max-w-4xl mx-auto space-y-6">

    <!-- Breadcrumb & Back -->
    <div class="flex items-center justify-between">
        <a href="/tenders" class="inline-flex items-center gap-1.5 text-xs font-semibold text-gem-700 hover:text
-gem-900 transition-colors">
            <svg class="w-4 h-4" fill="none" stroke="currentColor" viewBox="0 0 24 24"><path stroke-linecap="rou
nd" stroke-linejoin="round" stroke-width="2" d="M10 19l-7-7m0 0l7-7m-7 7h18"></path></svg>
            <span>Back to Active Tenders List</span>
        </a>
        <div class="text-[11px] font-mono text-slate-400">Ref: {{ tender.tender_id }}</div>
    </div>

    <!-- Tender Header Card -->
    <div class="bg-white rounded-2xl border border-slate-200/90 shadow-card p-6 sm:p-8 space-y-5">
        <div class="flex flex-wrap items-center gap-2">
            <span class="font-mono text-xs font-bold px-3 py-1 rounded-lg bg-gem-100 text-gem-900 border border-
gem-200">
                {{ tender.tender_id }}
            </span>
            <span class="text-xs font-semibold px-3 py-1 rounded-lg bg-slate-100 text-slate-700">
                {{ tender.department }}
            </span>
            <span class="text-xs font-bold px-2.5 py-0.5 rounded-full bg-emerald-100 text-emerald-800 border bor
der-emerald-200">
                ACTIVE
```

```
        </span>
    </div>

    <div>
        <h1 class="text-xl sm:text-2xl font-black text-slate-900 tracking-tight">
            {{ tender.title }}
        </h1>
        <p class="text-xs sm:text-sm text-slate-600 mt-2 leading-relaxed">
            {{ tender.description }}
        </p>
    </div>

    <!-- Key Procurement Metrics Grid -->
    <div class="grid grid-cols-1 sm:grid-cols-3 gap-4 pt-4 border-t border-slate-100">
        <div class="p-3.5 rounded-xl bg-slate-50 border border-slate-100">
            <div class="text-[10px] uppercase font-bold text-slate-400 tracking-wider">Estimated Tender Valu
e</div>
            <div class="text-base sm:text-lg font-black text-slate-900 font-mono mt-0.5">
                 {{ "{:,.2f}".format(tender.tender_value) }}
            </div>
        </div>

        <div class="p-3.5 rounded-xl bg-slate-50 border border-slate-100">
            <div class="text-[10px] uppercase font-bold text-slate-400 tracking-wider">Bid Submission Deadli
ne</div>
            <div class="text-base sm:text-lg font-black text-slate-900 font-mono mt-0.5">
                {{ tender.deadline[:10] }}
            </div>
        </div>

        <div class="p-3.5 rounded-xl bg-slate-50 border border-slate-100">
            <div class="text-[10px] uppercase font-bold text-slate-400 tracking-wider">Evaluation Framework<
/div>
            <div class="text-base sm:text-lg font-black text-emerald-700 font-mono mt-0.5">
                Automated AI
            </div>
        </div>
    </div>

    <!-- Apply CTA -->
    <div class="pt-4 flex flex-col sm:flex-row items-center justify-between gap-4 border-t border-slate-100"
>
        <div class="text-xs text-slate-500">
            Ensure all mandatory compliance documents are prepared in PDF format prior to submission.
        </div>
        <a href="/apply?tender_id={{ tender.id }}" class="w-full sm:w-auto px-6 py-3 bg-gem-700 hover:bg-gem
-800 text-white font-bold text-xs sm:text-sm rounded-xl shadow-md transition-all flex items-center justify-c
enter gap-2">
            <span>Proceed to Apply & Upload PDFs</span>
            <svg class="w-4 h-4" fill="none" stroke="currentColor" viewBox="0 0 24 24"><path stroke-linecap=
"round" stroke-linejoin="round" stroke-width="2" d="M14 5l7 7m0 0l-7 7m7-7H3"></path></svg>
        </a>
    </div>
</div>

<!-- Detailed Requirements Matrix -->
<div class="bg-white rounded-2xl border border-slate-200/90 shadow-card p-6 sm:p-8">
    <div class="flex items-center justify-between mb-4">
        <div>
            <h3 class="text-sm font-bold text-slate-900">Mandatory Eligibility & Technical Criteria</h3>
            <p class="text-xs text-slate-500">Evaluated deterministically by the GeM BidVerify AI Compliance
 Engine.</p>
        </div>
        <span class="text-xs font-mono font-bold text-gem-700 bg-gem-50 px-2.5 py-1 rounded-lg border border
-gem-200">
            {{ tender.requirements | length }} Statutory Rules
        </span>
    </div>

    <div class="divide-y divide-slate-100">
        {% for r in tender.requirements %}
        <div class="py-3.5 flex flex-col sm:flex-row sm:items-center justify-between gap-3">
            <div class="space-y-1">
                <div class="flex items-center gap-2">
                    <span class="font-mono text-[10px] font-bold px-2 py-0.5 rounded bg-slate-100 text-slate
-700">
                        {{ r.requirement_code }}
                    </span>
                    <span class="text-xs font-bold text-slate-900">
```

```
                        {{ r.requirement_name }}
                    </span>
                    {% if r.is_mandatory %}
                    <span class="text-[9px] font-bold px-1.5 py-0.5 rounded bg-rose-100 text-rose-800 border
     border-rose-200">
                        MANDATORY
                    </span>
                    {% endif %}
                </div>
                <div class="text-[11px] text-slate-500">
                    Rule Condition: <span class="text-slate-700 font-medium">{{ r.min_condition }}</span>
                </div>
            </div>

            <div class="flex items-center gap-3 text-right flex-shrink-0">
                <div class="text-[11px] text-slate-400 font-mono">
                    Verification: <span class="text-slate-700 font-semibold">{{ r.verification_source }}</sp
    an>
                </div>
                <div class="w-12 text-center text-xs font-mono font-black text-gem-900 bg-gem-50 py-1 rounde
    d border border-gem-200">
                    {{ r.weight }} pts
                </div>
            </div>
        </div>
        {% endfor %}
    </div>
    </div>

</div>
{% endblock %}


//// FILE: vendor_portal/templates/tenders.html  (169 lines)

{% extends "base.html" %}

{% block title %}Active Government Tenders - GeM Vendor Portal{% endblock %}

{% block content %}
<div class="space-y-6">

    <!-- Hero / Stats Banner -->
    <div class="bg-gradient-to-br from-gem-900 via-gem-800 to-gem-950 rounded-3xl text-white p-6 sm:p-8 shadow-x
    l relative overflow-hidden border border-gem-700/50">
        <div class="absolute -right-10 -bottom-10 w-72 h-72 bg-gem-500/10 rounded-full blur-3xl pointer-events-n
    one"></div>

        <div class="relative z-10 flex flex-col lg:flex-row lg:items-center justify-between gap-6">
            <div>
                <div class="inline-flex items-center gap-2 px-3 py-1 rounded-full bg-white/10 text-white text-xs
     font-semibold backdrop-blur-md mb-3 border border-white/15">
                    <span class="w-2 h-2 rounded-full bg-emerald-400 animate-pulse"></span>
                    Central Public Procurement Portal (CPPP) Feed
                </div>
                <h1 class="text-2xl sm:text-3xl font-extrabold tracking-tight">
                    Active Government Tenders & e-Bidding Matrix
                </h1>
                <p class="text-xs sm:text-sm text-gem-100/80 mt-1 max-w-2xl leading-relaxed">
                    Explore live procurements published by Central Ministries, State Departments, and Public Sec
    tor Undertakings. Select a tender to upload compliance documents for AI verification.
                </p>
            </div>

            <!-- Quick Action CTA -->
            <div class="flex items-center gap-3 self-start lg:self-auto flex-shrink-0">
                <a href="/apply" class="px-5 py-3 bg-gradient-to-r from-amber-500 to-saffron-500 hover:from-ambe
    r-600 hover:to-saffron-600 text-slate-950 font-extrabold text-xs sm:text-sm rounded-xl shadow-glow transitio
    n-all flex items-center gap-2">
                    <svg class="w-4 h-4" fill="none" stroke="currentColor" viewBox="0 0 24 24"><path stroke-line
    cap="round" stroke-linejoin="round" stroke-width="2" d="M12 4v16m8-8H4"></path></svg>
                    <span>Submit New Tender Bid</span>
                </a>
            </div>
        </div>

        <!-- Metric Counters Grid -->
        <div class="grid grid-cols-2 sm:grid-cols-4 gap-4 mt-8 pt-6 border-t border-white/10">
            <div>
```

```
            <div class="text-[11px] uppercase tracking-wider text-gem-200/70 font-semibold">Active Tenders</
div>
            <div class="text-2xl font-black text-white mt-0.5 font-mono">{{ tenders | length }}</div>
        </div>
        <div>
            <div class="text-[11px] uppercase tracking-wider text-gem-200/70 font-semibold">Total Scope Valu
e</div>
            <div class="text-2xl font-black text-amber-300 mt-0.5 font-mono"> {{ total_value_cr }} Cr</div>
        </div>
        <div>
            <div class="text-[11px] uppercase tracking-wider text-gem-200/70 font-semibold">AI Verification
Engine</div>
            <div class="text-2xl font-black text-emerald-400 mt-0.5 font-mono">GFR 149</div>
        </div>
        <div>
            <div class="text-[11px] uppercase tracking-wider text-gem-200/70 font-semibold">Verification Sys
tem</div>
            <div class="text-2xl font-black text-white mt-0.5 font-mono">Real-Time</div>
        </div>
    </div>
</div>

<!-- Search & Filter Controls -->
<div class="bg-white rounded-2xl border border-slate-200/90 shadow-card p-4 sm:p-5">
    <form action="/tenders" method="GET" class="flex flex-col sm:flex-row items-center gap-3">
        <div class="relative flex-1 w-full">
            <svg class="w-4 h-4 text-slate-400 absolute left-3.5 top-3.5" fill="none" stroke="currentColor"
viewBox="0 0 24 24"><path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M21 21l-6-6m2-5
a7 7 0 11-14 0 7 7 0 0114 0z"></path></svg>
            <input type="text" name="q" value="{{ query or '' }}"
                   placeholder="Search by Tender ID (e.g., GEM/2026/B/89201), scope keywords, cloud, AI "
                   class="w-full text-xs sm:text-sm pl-10 pr-4 py-2.5 border border-slate-300 rounded-xl foc
us:ring-2 focus:ring-gem-600 focus:border-gem-600 focus:outline-none placeholder:text-slate-400">
        </div>

        <div class="w-full sm:w-72">
            <select name="dept" onchange="this.form.submit()"
                    class="w-full text-xs sm:text-sm border border-slate-300 rounded-xl px-3.5 py-2.5 bg-sla
te-50 focus:bg-white focus:ring-2 focus:ring-gem-600 focus:outline-none">
                <option value="">All Ministries & Departments</option>
                {% for d in departments %}
                <option value="{{ d }}" {% if selected_dept == d %}selected{% endif %}>{{ d }}</option>
                {% endfor %}
            </select>
        </div>

        <button type="submit" class="w-full sm:w-auto px-5 py-2.5 bg-gem-700 hover:bg-gem-800 text-white tex
t-xs font-bold rounded-xl transition-all flex items-center justify-center gap-2">
            <span>Filter</span>
        </button>

        {% if query or selected_dept %}
        <a href="/tenders" class="w-full sm:w-auto px-3.5 py-2.5 text-xs font-semibold text-slate-500 hover:
text-slate-800 text-center">
            Clear
        </a>
        {% endif %}
    </form>
</div>

<!-- Tenders Matrix -->
<div class="space-y-4">
    {% if tenders %}
        {% for t in tenders %}
        <div class="bg-white rounded-2xl border border-slate-200/90 shadow-card hover:shadow-card-hover tran
sition-all p-5 sm:p-6 group">
            <div class="flex flex-col lg:flex-row lg:items-start justify-between gap-5">

                <!-- Left: Tender Details -->
                <div class="space-y-3 flex-1">
                    <div class="flex flex-wrap items-center gap-2">
                        <span class="font-mono text-xs font-black px-2.5 py-1 rounded-lg bg-gem-100 text-gem
-900 border border-gem-200">
                            {{ t.tender_id }}
                        </span>
                        <span class="text-[11px] font-semibold px-2.5 py-1 rounded-lg bg-slate-100 text-slat
e-700">
                            {{ t.department }}
                        </span>
```

```
                        <span class="text-[10px] font-bold px-2 py-0.5 rounded-full bg-emerald-100 text-emer
ald-800 border border-emerald-200">
                            OPEN FOR BIDDING
                        </span>
                    </div>

                    <div>
                        <h2 class="text-base sm:text-lg font-bold text-slate-900 group-hover:text-gem-800 tr
ansition-colors">
                            <a href="/tenders/{{ t.id }}">{{ t.title }}</a>
                        </h2>
                        <p class="text-xs text-slate-500 mt-1 line-clamp-2 leading-relaxed">
                            {{ t.description }}
                        </p>
                    </div>

                    <!-- Mandatory Compliance Requirements Badges -->
                    <div class="flex flex-wrap items-center gap-1.5 pt-1">
                        <span class="text-[10px] font-bold text-slate-400 uppercase tracking-wider mr-1">Com
pliance:</span>
                        {% for req in t.requirements %}
                        <span class="text-[10px] font-medium px-2 py-0.5 rounded-md {% if req.is_mandatory %
}bg-amber-50 text-amber-800 border border-amber-200{% else %}bg-slate-50 text-slate-600 border border-slate-
200{% endif %}">
                            {% if req.is_mandatory %}<span class="text-rose-500 mr-0.5">*</span>{% endif %}{
{ req.requirement_name }}
                        </span>
                        {% endfor %}
                    </div>
                </div>

                <!-- Right: Commercials & Apply CTA -->
                <div class="flex flex-row lg:flex-col items-center lg:items-end justify-between gap-4 pt-4 l
g:pt-0 border-t lg:border-t-0 border-slate-100 flex-shrink-0">
                    <div class="text-left lg:text-right">
                        <div class="text-[10px] text-slate-400 uppercase font-semibold">Estimated Tender Val
ue</div>
                        <div class="text-lg sm:text-xl font-extrabold text-slate-900 font-mono">
                             {{ "{:,.2f}".format(t.tender_value) }}
                        </div>
                        <div class="text-[11px] text-slate-500 mt-0.5 flex items-center lg:justify-end gap-1
">
                            <svg class="w-3.5 h-3.5 text-slate-400" fill="none" stroke="currentColor" viewBo
x="0 0 24 24"><path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M8 7V3m8 4V3m-9 8h10M
5 21h14a2 2 0 002-2V7a2 2 0 00-2-2H5a2 2 0 00-2 2v12a2 2 0 002 2z"></path></svg>
                            <span>Closing: <span class="font-semibold text-slate-700 font-mono">{{ t.deadlin
e[:10] }}</span></span>
                        </div>
                    </div>

                    <div class="flex items-center gap-2">
                        <a href="/tenders/{{ t.id }}" class="px-3.5 py-2 text-xs font-bold text-slate-700 ho
ver:text-gem-800 bg-slate-100 hover:bg-slate-200 rounded-xl transition-colors">
                            View Scope
                        </a>
                        <a href="/apply?tender_id={{ t.id }}" class="px-4 py-2 bg-gem-700 hover:bg-gem-800 t
ext-white text-xs font-bold rounded-xl shadow-sm hover:shadow transition-all flex items-center gap-1.5">
                            <span>Apply Now</span>
                            <svg class="w-3.5 h-3.5" fill="none" stroke="currentColor" viewBox="0 0 24 24"><
path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M14 5l7 7m0 0l-7 7m7-7H3"></path></s
vg>
                        </a>
                    </div>
                </div>

            </div>
        </div>
        {% endfor %}
    {% else %}
        <div class="bg-white rounded-2xl border border-slate-200 p-12 text-center">
            <div class="w-12 h-12 rounded-full bg-slate-100 text-slate-400 flex items-center justify-center
mx-auto mb-3">
                <svg class="w-6 h-6" fill="none" stroke="currentColor" viewBox="0 0 24 24"><path stroke-line
cap="round" stroke-linejoin="round" stroke-width="2" d="M9.172 16.172a4 4 0 015.656 0M9 10h.01M15 10h.01M21
12a9 9 0 11-18 0 9 9 0 0118 0z"></path></svg>
            </div>
            <h3 class="text-sm font-bold text-slate-800">No active tenders matched your criteria</h3>
            <p class="text-xs text-slate-500 mt-1">Try resetting your search query or department filter.</p>
            <a href="/tenders" class="inline-block mt-4 px-4 py-2 bg-gem-700 text-white text-xs font-bold ro
```

```
    unded-xl">View All Active Tenders</a>
            </div>
        {% endif %}
    </div>

</div>
{% endblock %}


//// FILE: vite.config.js  (13 lines)

import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
import { viteSingleFile } from 'vite-plugin-singlefile';

// `npm run dev`   -> dev server with hot reload; requests to /api are forwarded to the backend on port 8000
// `npm run build` -> dist/index.html as ONE self-contained file
const backend = 'http://127.0.0.1:8000';

export default defineConfig({
  plugins: [react(), viteSingleFile()],
  server: { port: 5173, proxy: { '/api': { target: backend, changeOrigin: true } } },
  preview: { proxy: { '/api': { target: backend, changeOrigin: true } } },
});
```